

TP 02 - Todo List

Ajoutez les fonctionnalités demandées dans l'application existante de fin de jour 1, sans repartir de zéro.

Travail demandé

- Ajouter une tâche avec un **formulaire contrôlé**.
- Modifier une tâche **directement dans la liste**.
- Marquer une tâche comme terminée / non terminée.
- Supprimer une tâche.
- Afficher un **compteur** de tâches restantes.
- Filtrer l'affichage : **toutes, actives, terminées**.

Point de départ

Ajoutez ce state, ou un équivalent, dans votre application existante.

```
type Todo = {
  id: number;
  title: string;
  done: boolean;
};

const [todos, setTodos] = useState<Todo[]>([
  { id: 1, title: "Apprendre React", done: false },
  { id: 2, title: "Créer un composant", done: true },
  { id: 3, title: "Maîtriser les hooks", done: false },
]);

const [filter, setFilter] = useState<"all" | "active" | "done">("all");
const [newTitle, setNewTitle] = useState("");
```

Fonctionnalités à implémenter

1. Ajouter une tâche

Prévoir un champ texte contrôlé et un bouton d'ajout. Une tâche vide ne doit pas être ajoutée. Après ajout, le champ doit être vidé.

2. Changer l'état terminé / non terminé

Chaque tâche doit pouvoir être cochée ou décochée. L'état **done** doit être mis à jour correctement.

3. Supprimer une tâche

Chaque tâche doit posséder une action de suppression. La suppression doit retirer immédiatement la tâche de la liste.

4. Modifier une tâche inline

Prévoir un mode édition directement dans la liste. Une valeur vide ne doit pas remplacer le titre existant.

5. Afficher le nombre de tâches restantes

Afficher clairement le nombre de tâches non terminées.

6. Filtrer les tâches

Permettre d'afficher toutes les tâches, seulement les actives ou seulement les terminées. Le filtre modifie l'affichage, pas les données source.

Données dérivées attendues

Le filtre et le compteur doivent être calculés à partir du tableau principal.

```
const filteredTodos = todos.filter((todo) => {
  if (filter === "active") return !todo.done;
  if (filter === "done") return todo.done;
  return true;
});

const remaining = todos.filter((todo) => !todo.done).length;
```

Contraintes

- Ne repartez pas de zéro.
- Ne créez pas une nouvelle application à côté.
- Conservez la structure globale du projet existant autant que possible.
- Restez en **React + TypeScript** avec des fichiers **.tsx**.
- Ne modifiez jamais directement le tableau **todos** ni les objets qu'il contient.
- Utilisez des mises à jour immuables avec **map**, **filter** et l'opérateur **...**.

Actions CRUD attendues

- **addTodo** : ajoute une nouvelle tâche avec un identifiant unique, un titre nettoyé avec **trim()** et **done** à **false**.
- **toggleTodo** : inverse la valeur **done** de la tâche ciblée.
- **deleteTodo** : retire la tâche correspondant à l'identifiant fourni.
- **editTodo** : remplace le titre d'une tâche par un nouveau titre valide.

Exemples de logique attendue

```
const addTodo = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  if (!newTitle.trim()) return;

  setTodos((prev) => [
    ...prev,
    { id: Date.now(), title: newTitle.trim(), done: false }
  ]);

  setNewTitle("");
};

const toggleTodo = (id: number) =>
  setTodos((prev) =>
    prev.map((t) => (t.id === id ? { ...t, done: !t.done } : t))
  );

const deleteTodo = (id: number) =>
  setTodos((prev) => prev.filter((t) => t.id !== id));

const editTodo = (id: number, title: string) =>
  setTodos((prev) =>
    prev.map((t) => (t.id === id ? { ...t, title } : t))
  );
```

Validation

- L'application existante fonctionne toujours.
- L'ajout, la modification, le toggle, la suppression, le compteur et le filtre fonctionnent.
- Le code compile sans erreur TypeScript.
- Aucune mutation directe du state n'est utilisée.